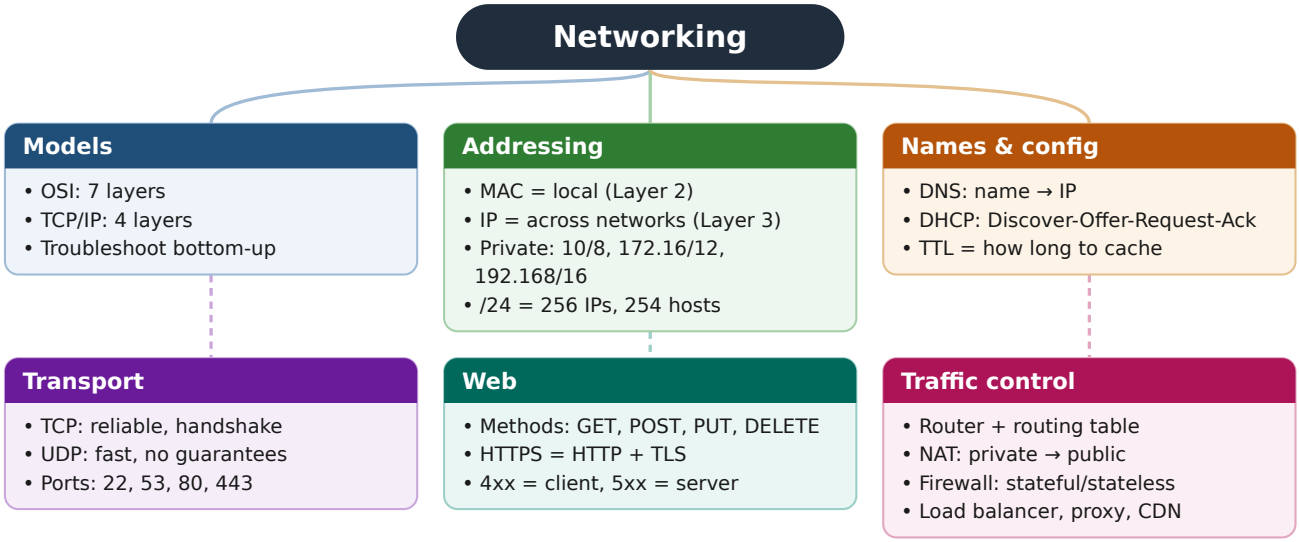
2. Networking

DevOps, Cloud & Platform Engineering Notes · Section 2 of 18 · Thirupathi Keloth · v1.4

In this section: 2.1 How data travels · 2.2 OSI and TCP/IP models · 2.3 Devices and addresses · 2.4 IPv4 addressing · 2.5 Subnets and CIDR · 2.6 DNS (Domain Name System) · 2.7 DHCP · 2.8 Ports, TCP and UDP · 2.9 HTTP and HTTPS · 2.10 Routing, NAT, firewalls, VPN · 2.11 Load balancers, proxies, CDN · 2.12 SSH · 2.13 Networking commands · 2.14 Network troubleshooting order



2.1 How data travels

```
Client  ──request──►  Router  ──►  ISP  ──►  Internet  ──►  Server
        ◄──response──
```

- Data is split into small pieces called **packets**. Each packet carries a source and destination IP address.
- **Protocol** = an agreed set of rules for communicating (HTTP, TCP, DNS...).

2.2 OSI and TCP/IP models

OSI layer	TCP/IP layer	Job	Examples
7 Application	Application	What the user or app wants	HTTP, HTTPS, DNS, SSH, FTP
6 Presentation	Application	Format and encryption	TLS, JSON
5 Session	Application	Keeping conversations open	Sessions
4 Transport	Transport	Reliable or fast delivery, ports	TCP, UDP
3 Network	Internet	Addressing and routing	IP, ICMP, routers
2 Data link	Network access	Delivery inside the local network	MAC, Ethernet, switches, ARP
1 Physical	Network access	Bits on a cable or radio	Cables, Wi-Fi

Troubleshoot bottom-up: cable/Wi-Fi → IP address → gateway → DNS → port → application.

2.3 Devices and addresses

Term	Meaning
MAC address	A physical address burned into the network card, e.g. `00:1A:2B:3C:4D:5E`. Used inside the local network
IP address	A logical address used to reach a device across networks
Switch	Connects devices inside one network using MAC addresses
Router	Connects different networks and forwards packets using IP addresses
Default gateway	The router your device sends traffic to when the destination is outside your network
ARP	Finds the MAC address that belongs to an IP address on the local network

2.4 IPv4 addressing

- 32 bits, written as 4 numbers (0–255): `192.168.1.10`
- **Private ranges** (not reachable from the internet): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`
- **Public IP:** unique on the internet, assigned by your ISP or the cloud provider
- **Special:** `127.0.0.1` = loopback (this machine, "localhost"); `169.254.x.x` = APIPA (DHCP failed); `0.0.0.0` = all addresses
- **IPv6:** 128 bits, e.g. `2001:db8::1`. Created because IPv4 ran out of addresses

Binary refresher (one octet):

Bit value	128	64	32	16	8	4	2	1
192 =	1	1	0	0	0	0	0	0

2.5 Subnets and CIDR

- A **subnet** is a smaller network carved out of a bigger one.
- **CIDR** `/n` = how many bits belong to the network part. The rest is for hosts.
- **Usable hosts** = $2^{(32 - n)} - 2$ (network address and broadcast address are reserved). AWS reserves 5 per subnet.

CIDR	Subnet mask	Total IPs	Usable hosts
/8	255.0.0.0	16,777,216	16,777,214
/16	255.255.0.0	65,536	65,534
/20	255.255.240.0	4,096	4,094
/24	255.255.255.0	256	254
/26	255.255.255.192	64	62
/28	255.255.255.240	16	14
/32	255.255.255.255	1	1 (a single host)

Example: `10.0.1.0/24` → network `10.0.1.0`, hosts `10.0.1.1–10.0.1.254`, broadcast `10.0.1.255`.

Typical cloud plan: VPC `10.0.0.0/16` → public subnets `10.0.1.0/24`, `10.0.2.0/24` → private subnets `10.0.11.0/24`, `10.0.12.0/24`.

How subnetting works (binary method)

Place values of one octet (memorize): 128 · 64 · 32 · 16 · 8 · 4 · 2 · 1. All eight bits on = 255.

Convert	Method	Example
Binary → decimal	Add the place values where the bit is 1	`10101100` = 128+32+8+4 = **172**
Decimal → binary	Left to right: if the number ≥ place value, write 1 and subtract; else 0	200 → 128 ✓ (72), 64 ✓ (8), 32 ✗, 16 ✗, 8 ✓ (0), 4 ✗, 2 ✗, 1 ✗ = `11001000`

The mask in binary: 1 = network bit, 0 = host bit. `/27` = `11111111.11111111.11111111.11100000` = 255.255.255.224 → 5 host bits → 2^5 = 32 addresses, 30 usable. **Network address** = IP AND mask (host bits set to 0). **Broadcast** = all host bits set to 1.

Fast block-size method:

1. Find the octet where the mask is not 255 or 0.
2. Block size = 256 − mask value in that octet.
3. Subnets start at multiples of the block size. The IP belongs to the largest multiple ≤ its value.
4. Broadcast = next subnet − 1.

Example `192.168.10.77/27`: mask .224 → block 32 → subnets .0, .32, **.64**, .96 → network `192.168.10.64`, hosts `.65`–`.94`, broadcast `.95`. **Hosts → prefix:** ≤14 → /28 · ≤30 → /27 · ≤62 → /26 · ≤126 → /25 · ≤254 → /24 · ≤510 → /23.

(Full step-by-step binary and subnetting practice: see the *Computer Networking* notes, section 6.)

2.6 DNS (Domain Name System)

Turns names into IPs: `google.com → 142.250.x.x`.

```
Browser → OS cache → Resolver (ISP / 8.8.8.8)
        → Root (.) → TLD (.com) → Authoritative server → IP returned
```

Record	Purpose
A	Name → IPv4 address
AAAA	Name → IPv6 address
CNAME	Name → another name (alias)
MX	Mail server for the domain
TXT	Text, e.g. domain verification, SPF
NS	Which name servers are authoritative for the domain
TTL	How long (in seconds) a record may be cached

`/etc/hosts` (Linux) or `C:\Windows\System32\drivers\etc\hosts` (Windows) overrides DNS locally.

2.7 DHCP

Gives devices an IP address, subnet mask, gateway and DNS server automatically. Process **DORA**: Discover → Offer → Request → Acknowledge.

2.8 Ports, TCP and UDP

- **Port** = a numbered door on a computer for a specific service (0–65535). **Socket** = IP + port, e.g. `10.0.1.5:443`.

	TCP	UDP
Connection	Yes (3-way handshake)	No
Reliability	Guaranteed, ordered, retransmits	Best effort
Speed	Slower	Faster
Used for	Web, SSH, databases, email	DNS, video calls, streaming, gaming

TCP 3-way handshake: `SYN → SYN-ACK → ACK`

Port	Service	Port	Service
22	SSH	443	HTTPS
25	SMTP (mail)	3306	MySQL
53	DNS	5432	PostgreSQL
80	HTTP	6379	Redis
123	NTP (time)	8080	Common app/Jenkins port
3389	RDP (Windows remote)	9090 / 3000	Prometheus / Grafana

2.9 HTTP and HTTPS

Method	Use	Status code	Meaning
GET	Read data	200	OK
POST	Create data	201	Created
PUT / PATCH	Update data	301 / 302	Redirect
DELETE	Delete data	400	Bad request
		401 / 403	Not logged in / not allowed
		404	Not found
		500	Server error
		502 / 503 / 504	Bad gateway / unavailable / timeout (often a backend or load balancer problem)

- **Headers** carry extra info (Host, Content-Type, Authorization, Cookie).
- **HTTPS** = HTTP encrypted with **TLS**. The server proves its identity with a **certificate** signed by a trusted **Certificate Authority (CA)**.
- **REST API** = a server that accepts HTTP requests and returns data, usually JSON.

2.10 Routing, NAT, firewalls, VPN

Term	Meaning
Routing table	A list of "to reach network X, send to Y". `0.0.0.0/0` = default route (everything else)
Hop	Each router a packet passes through
NAT	Translates private IPs to a public IP so private devices can reach the internet. Your home router does this
Firewall	Allows or blocks traffic by IP, port and protocol
Stateful firewall	Remembers connections; return traffic is allowed automatically (AWS Security Groups)
Stateless firewall	Checks every packet; return traffic needs its own rule (AWS NACLs)
VPN	An encrypted tunnel between networks or from a user to a network

2.11 Load balancers, proxies, CDN

Term	Meaning
Load balancer	Spreads traffic across many servers and sends traffic only to healthy ones (health checks)
L4 load balancer	Works on IP and port (TCP/UDP). Fast. AWS NLB
L7 load balancer	Understands HTTP: routes by path or host. AWS ALB
Forward proxy	Sits in front of **clients** and makes requests for them (company web filter)
Reverse proxy	Sits in front of **servers** and receives requests for them (nginx, ALB)
CDN	Caches content in many locations close to users (CloudFront, Cloudflare)

2.12 SSH

Secure Shell: encrypted remote login to a server on port 22. Uses a **key pair**: the private key stays on your machine and must never be shared; the public key goes on the server.

2.13 Networking commands

Windows	Linux	Purpose
`ipconfig /all`	`ip a`	Show IP and MAC addresses
`ping host`	`ping -c 4 host`	Test reachability (ICMP)
`tracert host`	`traceroute host` / `mtr host`	Show each hop to the destination
`nslookup name`	`dig name` / `dig +short name`	DNS lookup
`route print`	`ip route`	Show the routing table
`arp -a`	`ip neigh`	Show the ARP table
`netstat -ano`	`ss -tulpn`	Show listening ports and connections

Windows	Linux	Purpose
`Test-NetConnection host -Port 443`	`nc -zv host 443`	Test whether a port is open
`curl.exe -v https://site`	`curl -v https://site`	Make an HTTP request and see the details
`ipconfig /flushdns`	`resolvectl flush-caches`	Clear the DNS cache
—	`tcpdump -i any port 53`	Capture packets

Wireshark = a graphical packet capture tool. Useful filters: `icmp`, `dns`, `tcp.port == 443`, `ip.addr == 10.0.0.5`.

2.14 Network troubleshooting order

1. Is there a link? (Wi-Fi / cable, `ip a`)
2. Do I have a valid IP? (not `169.254.x.x`)
3. Can I reach the gateway? (`ping <gateway>`)
4. Can I reach the internet by IP? (`ping 8.8.8.8`)
5. Does DNS work? (`nslookup google.com`)
6. Is the port open? (`Test-NetConnection` / `nc -zv`)
7. Does the application answer? (`curl -v`)
8. Is a firewall or security group blocking it?

> **REMEMBER**
>
> - OSI layers 7 → 1: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.
> - Status codes: **2xx** OK · **3xx** moved · **4xx** you made a mistake · **5xx** the server made a mistake.
> - Troubleshoot: link → IP → gateway → internet (8.8.8.8) → DNS → port → application.
> - Subnet hosts = $2^{\text{host bits}} - 2$. Block size = 256 − mask octet.